

Appointment Chatbot

Module Steps Reference

Module 1: RAG Pipeline

Estimate Agent — answer pricing & prerequisites questions from business documents

INDEXING (runs once at startup)

- Step 1:** Read documents from configured path (.txt, .pdf, .docx files in Documents/ folder)
- Step 2:** Extract raw text per file type: .txt via File.ReadAllText | .pdf via PdfPig | .docx via OpenXml
- Step 3:** Chunk text — split on blank lines so each service entry becomes one searchable chunk
- Step 4:** Embed each chunk — send to Azure OpenAI text-embedding-3-small, get back float[1536]
- Step 5:** Store in memory — List { Text, SourceFile, Embedding }

QUERYING (runs on every customer service question)

- Step 6:** Receive customer query e.g. 'how much does a blood test cost?'
- Step 7:** Embed the query using the same Azure OpenAI embeddings API
- Step 8:** Cosine similarity — compare query vector against every stored chunk vector
- Step 9:** Return top N chunks by score (highest score = most semantically relevant)
- Step 10:** Pass retrieved chunks to LLM as context — LLM answers accurately from real data

Module 2: Multi-Agent Orchestration

Semantic Kernel — route messages to specialist agents and take real-world actions

CONVERSATION FLOW (every user message)

- Step 1:** User sends message to POST /api/chat
- Step 2:** ChatController passes message + history to IChatOrchestrator
- Step 3:** Orchestrator rebuilds ChatHistory from previous turns, adds new user message
- Step 4:** Passes to Semantic Kernel ChatCompletionAgent with 5 plugins registered as tools
- Step 5:** LLM decides which tool to call first (e.g. get_service_estimate for pricing questions)
- Step 6:** Semantic Kernel intercepts tool call, executes real C# method, returns result to LLM
- Step 7:** LLM processes result, decides next action or responds to user
- Step 8:** Loop continues until LLM produces a final text response
- Step 9:** Orchestrator returns reply + updated history to controller
- Step 10:** Controller sends ChatResponse back to widget

Module 3: Google Calendar Integration

Check availability, book, reschedule and cancel real calendar events

SETUP (one-time per business)

- Step 1:** Business owner creates Google Cloud project and enables Google Calendar API
- Step 2:** Creates service account and downloads JSON key file
- Step 3:** Shares their Google Calendar with the service account email (Make changes to events)
- Step 4:** JSON key path + CalendarId added to appsettings.Development.json

CHECKING AVAILABILITY

- Step 1:** Receive requested date
- Step 2:** Calculate business hours window (9am-5pm)
- Step 3:** Call Google FreeBusy API — returns blocked time ranges for that calendar

Step 4: Walk every 30-min slot, skip any overlapping busy ranges, return free slots

BOOKING

Step 1: Verify slot still available (race condition protection)

Step 2: Build Event object — summary, description, start/end, attendees

Step 3: Call Google Calendar Events.Insert API

Step 4: Google creates event and sends calendar invites to all attendees

Step 5: Return eventId — saved to IBookingStore for reschedule/cancel lookup

RESCHEDULING

Step 1: Look up existing booking by customer email from IBookingStore — get eventId

Step 2: Verify new slot is available

Step 3: Fetch existing event via Events.Get

Step 4: Update start/end times, call Events.Update — Google sends update notifications

Step 5: Update IBookingStore with new times

Module 4: Outlook Calendar Integration

Same interface as Google Calendar, powered by Microsoft Graph API

SETUP (one-time per business)

Step 1: Register app in Azure Portal (App Registration)

Step 2: Add Calendars.ReadWrite permission via Microsoft Graph

Step 3: Create client secret — note Tenant ID, Client ID, Client Secret

Step 4: Add credentials to appsettings.Development.json

AUTHENTICATION (automatic before each API call)

Step 1: Azure.Identity reads TenantId + ClientId + ClientSecret from config

Step 2: Calls Microsoft login endpoint to fetch access token (valid 1 hour)

Step 3: Microsoft.Graph uses token for all subsequent API calls

Step 4: Token auto-refreshes before expiry — transparent to the rest of the app

BOOKING / AVAILABILITY

Step 1: Check availability — POST /users/{userId}/calendar/getSchedule

Step 2: Same slot-walking logic as Google Calendar

Step 3: Book — POST /users/{userId}/calendar/events

Step 4: Microsoft creates event and sends Outlook calendar invites to attendees

Module 5: Staff Store

Look up provider email by name from a CSV or Excel file — no code changes to add staff

LOADING (at startup + file change)

Step 1: Read StaffFilePath from BusinessConfig in appsettings.json

Step 2: Detect file type — .csv or .xlsx

Step 3: Parse: CSV via line-by-line split | Excel via ExcelDataReader rows

Step 4: Build Dictionary in memory

QUERYING (when booking needs a provider)

Step 1: Customer names a provider e.g. 'book with Dr Smith'

Step 2: Agent calls GetProviderEmail('Dr Smith')

Step 3: Dictionary lookup returns 'drsmith@clinic.com'

Step 4: Email passed to BookingPlugin as providerEmail

Module 6: Evening Reminder Service

Background service — sends owner a summary of tomorrow's appointments at 6pm

SCHEDULING (runs continuously)

- Step 1:** IHostedService starts with the app
- Step 2:** Checks current time every minute
- Step 3:** When time matches ReminderTime config (e.g. 18:00) — runs reminder job
- Step 4:** Sets 'sent today' flag to avoid duplicates, resets at midnight

REMINDER JOB (runs once at configured time)

- Step 1:** Calculate tomorrow's date
 - Step 2:** Call ICalendarService.GetEventsForDateAsync(tomorrow)
 - Step 3:** For each event — extract customer name, time, service type
 - Step 4:** RAG search prerequisites for each service from document index
 - Step 5:** Build formatted email body with appointment details + prerequisites
 - Step 6:** Send via IEmailService to BusinessConfig.OwnerEmail
-

Module 7: Embeddable Widget

Any business website embeds the chatbot with one script tag — no framework required

EMBEDDING (business does once)

- Step 1:** Business adds one script tag to their website with data-api-url and data-accent
- Step 2:** Script loads from your server — no npm, no build step, no framework

WIDGET INITIALISATION (on page load)

- Step 1:** Script reads data-api-url and data-accent from the script tag attributes
- Step 2:** Injects CSS styles into page head (scoped, won't clash with client's styles)
- Step 3:** Creates fixed chat bubble button (bottom-right corner)
- Step 4:** Creates hidden chat panel with header, message area, and input row
- Step 5:** Attaches click and keydown event listeners

CONVERSATION (every customer message)

- Step 1:** Customer clicks bubble — panel opens, greeting shown
 - Step 2:** Customer types and sends message
 - Step 3:** Widget shows Typing... indicator immediately
 - Step 4:** POST /api/chat with { message, history } — history carries conversation context
 - Step 5:** Receives { reply, history } response
 - Step 6:** Removes typing indicator, displays reply, saves updated history in memory
-

Module 8: JWT Authentication (Later Step)

Secure the API so only authorized widgets can call it

TOKEN ISSUANCE

- Step 1:** Widget requests session token: POST /api/auth/token { businessId }
- Step 2:** Server validates businessId exists in config
- Step 3:** Server generates JWT: header + payload (businessId, sessionId, expiry) + HMACSHA256 signature
- Step 4:** Returns signed token to widget

TOKEN USAGE

- Step 1:** Widget stores token in memory (not localStorage)
- Step 2:** Every /api/chat request includes Authorization: Bearer {token} header

Step 3: JWT middleware validates signature and expiry on every request

Step 4: Returns 401 Unauthorized if token missing, invalid, or expired

TOKEN REFRESH

Step 1: Widget checks token expiry before each request

Step 2: If less than 5 minutes remaining — requests new token automatically

Step 3: New token issued, old one discarded — seamless to the user

Module 9: Serilog Structured Logging (Later Step)

Replace default ASP.NET logging with structured, queryable logs

SETUP

Step 1: Add Serilog NuGet packages (Serilog.AspNetCore, sinks for Console + File)

Step 2: Configure sinks and minimum log levels in appsettings.json

Step 3: Replace default ASP.NET ILogger with Serilog in Program.cs

WHAT GETS LOGGED

Step 1: Every /api/chat request — timestamp, businessId, messageLength, sessionId

Step 2: Every agent tool call — agentName, toolName, inputParams, duration

Step 3: Every calendar API call — provider, operation, success/fail, duration

Step 4: Every email sent — recipientEmail (masked), emailType

Step 5: Every error — exception type, stack trace, request context

STRUCTURED FORMAT (queryable, not plain text)

Step 1: Each log entry is a structured object with named properties

Step 2: e.g. { Event: AppointmentBooked, CustomerEmail: j***@x.com, Duration: 234ms }

Step 3: Can be queried/filtered by field — much more useful than plain text logs

Each module maps to a service interface — swap implementations via appsettings.json without code changes.